

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

DEVROOM (Live Coding Platform)

Version 1.0

SHAFIA ARIF (CS-23107)

SANIA SHAKEEL (CS-23108)

ESHA FAROOQ (CS-23112)

*Department of Computer and Information Systems
NED University of Engineering and Technology*

Submitted to
Ms. Fakhra Aftab

October 19, 2025

Contents

1	Introduction	3
1.1	Introduction	3
1.2	Document Purpose	3
1.3	Product Scope	3
1.4	Intended Audience and Document Overview	4
1.5	Definitions, Acronyms and Abbreviations	4
1.6	Document Conventions	5
1.7	References and Acknowledgments	5
2	Overall Description	6
2.1	Product Perspective	6
2.2	Product Functionality	7
2.3	Users and Characteristics	8
2.3.1	System Administrator	8
2.3.2	Instructor	8
2.3.3	Student	8
2.4	Operating environment	8
2.5	Design and Implementation Constraints	8
2.5.1	Hardware Constraint	8
2.5.2	Browser Constraint	9
2.5.3	JavaScript Constraint	9
2.5.4	Internet Constraint	9
2.5.5	Input Device Constraint	9
2.5.6	Performance Constraint	9
2.5.7	Storage Constraint	9
2.5.8	Scalability Constraint	9
2.5.9	Compatibility Constraint	9
2.5.10	User Documentation	10
2.6	Assumptions and Dependencies	10
2.6.1	Assumptions	10
2.6.2	Dependencies	10
3	Specific Requirements	11
3.1	External Interface Requirements	11
3.1.1	User Interfaces	11
3.1.2	Hardware Interfaces	12
3.1.3	Software Interfaces	13
3.1.4	Communications Interfaces	13

3.2	Functional requirements	13
3.2.1	Main Page	13
3.2.2	Code Editor Page	14
3.2.3	Save and Load Session	14
3.2.4	Download Code	14
3.2.5	Chat Functionality	15
3.3	Behaviour Requirements	17
3.3.1	Use case View	17
4	Other Non-Functional Requirements	19
4.1	Performance Requirements	19
4.2	Safety and Security Requirements	19
4.2.1	Safety	19
4.2.2	Security	20
4.3	Software Quality Attributes	20
4.3.1	Adaptibility	20
4.3.2	Availability	20
4.3.3	Correctness	21
4.3.4	Flexibility	21
4.3.5	Interoperability	21
4.3.6	Maintainability	21
4.3.7	Portability	21
4.3.8	Reliability	21
4.3.9	Reuseability	21
4.3.10	Robustness	21
4.3.11	Testability	22
4.3.12	Useability	22
5	Other Requirements	23
6	Appendix A - Data Dictionary	24
6.1	System Constants	24
6.2	State Variables	25
6.3	Input Variables	26
6.4	Output Variables	27
6.5	Functional Data (Key Operations)	28
6.6	Notes	28

Chapter 1

Introduction

1.1 Introduction

The **DevRoom** is a real-time collaborative code editing platform that allows multiple users to work together within a shared coding environment. It provides an interactive interface where users can write, edit, and view code simultaneously, promoting teamwork and learning through live collaboration. The system focuses on real-time synchronization, communication, and ease of use, making it ideal for students and developers who wish to code together remotely. In this section, a brief overview of the software, including its scope, intended users, core technologies, and overall purpose, is presented to provide a clear understanding of the project objectives and functionalities.

1.2 Document Purpose

This document describes the requirements and specifications of the **DevRoom** project. It explains the functional and non-functional features of the system, including real-time code synchronization, collaborative editing, chat communication, and session management. The main purpose of this web application is to provide a shared coding environment that enables multiple users to write and edit code simultaneously over the internet. It aims to facilitate teamwork, learning, and remote collaboration by offering an interactive, user-friendly, and responsive platform for developers and students to code together effectively.

1.3 Product Scope

DevRoom is a real-time collaborative code editing platform that enables multiple users to write, edit, and communicate within a shared coding environment. This web application serves as an effective tool for both educators and students to engage in collaborative coding exercises, online coding sessions, and virtual programming workshops. Instructors can use the platform to conduct live demonstrations, while students can work together on assignments and projects remotely. Due to its user-friendly and intuitive interface, **DevRoom** can also be used independently by individuals for pair programming or group learning. The main objective of developing this system is to promote collaborative learning and enhance coding efficiency through real-time interaction and teamwork.

1.4 Intended Audience and Document Overview

This document is intended for students, instructors, and software developers who wish to understand, use, or evaluate the **DevRoom** system. It can be used as a reference guide in universities, programming courses, or collaborative learning environments to study the system's design, features, and technical specifications. The document will also assist evaluators and developers in understanding the functional and non-functional aspects of the software for further enhancement or maintenance.

The rest of this SRS is organized as follows:

- **Section 2** provides an overall description of the system, including its functions, constraints, assumptions, and dependencies.
- **Section 3** details the system features and specific functional requirements, including user interactions and use case diagrams.
- **Section 4** outlines the non-functional requirements such as performance, security, and usability considerations, along with software quality attributes.

Each section contributes to a comprehensive understanding of **DevRoom**, describing its functionality and behavior from both a user and developer perspective.

1.5 Definitions, Acronyms and Abbreviations

- **API:** Application Programming Interface – A set of functions and protocols that allow different software components to communicate with each other.
- **CRUD:** Create, Read, Update, Delete – The four basic operations of persistent storage used in database systems.
- **CSS:** Cascading Style Sheets – A stylesheet language used for describing the presentation of a document written in HTML.
- **DevRoom:** The proposed real-time collaborative code editing platform that enables multiple users to write and edit code simultaneously within a shared workspace.
- **DOM:** Document Object Model – A programming interface for web documents that represents the structure of a webpage.
- **Express.js:** A lightweight Node.js framework used to build web applications and APIs.
- **GUI:** Graphical User Interface – A visual interface that allows users to interact with the system through graphical elements.
- **HTML:** Hypertext Markup Language – The standard markup language used to create and design web pages.
- **IDE:** Integrated Development Environment – A software application that provides comprehensive facilities for software development.
- **JS (JavaScript):** A high-level, interpreted programming language used to create interactive effects and dynamic web content.

- **MongoDB:** A NoSQL database used to store data in a flexible, JSON-like format.
- **Node.js:** A runtime environment that allows the execution of JavaScript code on the server side.
- **Socket.IO:** A JavaScript library that enables real-time, bidirectional communication between web clients and servers..
- **SRS:** Software Requirements Specification – A document that describes what the software will do and how it is expected to perform.
- **UI:** User Interface – The part of a software system that users interact with directly.
- **UX:** User Experience – The overall experience and satisfaction a user feels while interacting with a software product.
- **VS Code:** Visual Studio Code – A source-code editor used for software development.
- **WebApp:** Web Application – A client–server software program that runs in a web browser and interacts with a backend server.

1.6 Document Conventions

This Software Requirements Specification (SRS) document for **DevRoom** has been prepared using standard software engineering documentation guidelines. The document follows a structured format with numbered sections and subsections for clarity and consistency. It uses clear, descriptive headings to represent each component of the system. Technical terms, acronyms, and abbreviations are defined in Section 1.4. The document has been written using **LaTeX**, a document preparation system that allows the creation of well-formatted, professional technical documents in plain text format.

1.7 References and Acknowledgments

- IEEE. *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. IEEE Computer Society, 1998.
- Braude, Eric J. *Software Engineering: An Object-Oriented Perspective*. Wiley, 2001.
- Reaves, SPMP. *Software Project Management Plan – Jacksonville State University Computing and Information Sciences Web Accessible Alumni Database*. Jacksonville State University, 2003.
- Official Documentation: *Node.js*, *Express.js*, *Socket.IO*, and *MongoDB* – accessed from their respective developer guides.
- The authors would like to acknowledge the guidance of the course instructor and the resources provided by the university, which contributed to the successful preparation of this Software Requirements Specification for **DevRoom**.

Chapter 2

Overall Description

2.1 Product Perspective

DevRoom is a new, self-contained web-based system designed to provide a real-time collaborative coding environment for developers, students, and instructors. Unlike traditional code editors that are limited to single-user functionality, **DevRoom** enables multiple users to write, edit, and communicate simultaneously within the same coding session. It integrates live synchronization, syntax highlighting, and a built-in chat interface to create an interactive and engaging workspace. The system operates as a standalone web application, accessible through any modern browser, and requires only an internet connection to function.

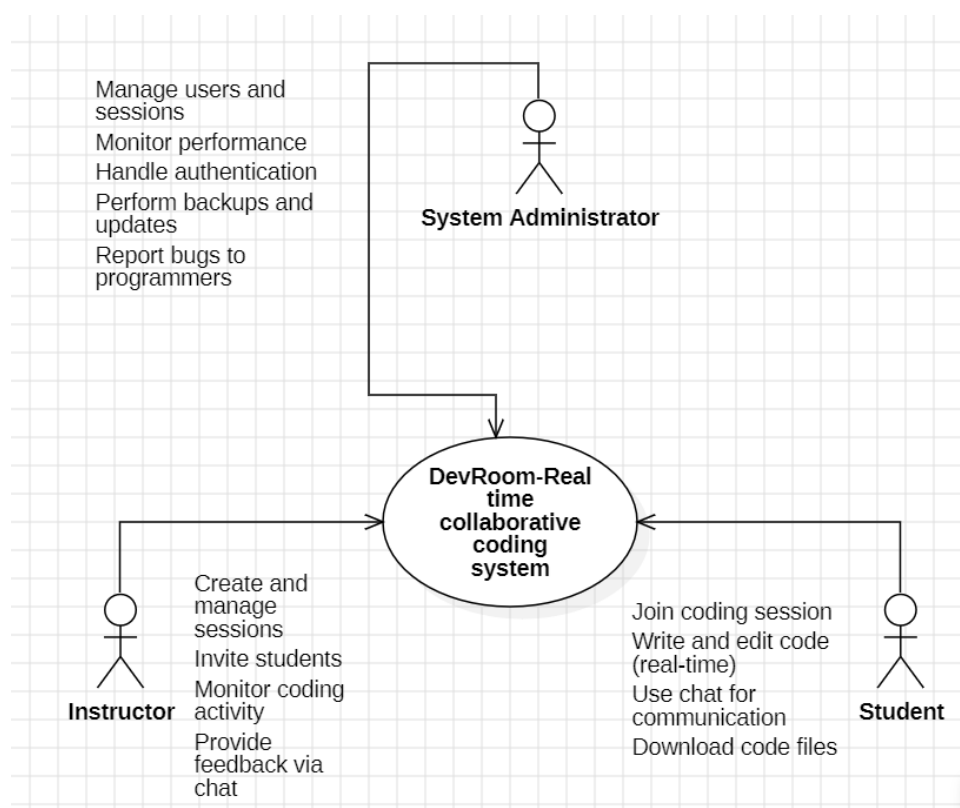


Figure 2.1: System Architecture Diagram

2.2 Product Functionality

The DevRoom web application provides a set of core functions that enable real-time, collaborative code editing and communication among multiple users. These functions collectively define how users interact with the system and how the system manages collaboration, communication, and data storage.

The major functionalities of DevRoom are summarized as follows:

- Allow users to create a new collaborative coding session or join an existing one using a unique room link or session ID.
- Enable multiple users to write and edit code simultaneously, with real-time synchronization across all connected clients.
- Provide syntax highlighting and support for multiple programming languages to improve readability and understanding.
- Allow users to communicate through an integrated chat box for discussion and coordination during coding sessions.
- Automatically update all participants' code views when any user makes changes in the shared workspace.
- Offer the ability to save, load, or restore previous coding sessions stored in the database.
- Notify users when new participants join or leave a session.
- Display connection status and handle disconnections gracefully without data loss.
- Enable users to download their code files locally for offline use.
- Allow instructors or session hosts to control room access (create, close, or manage sessions).

These functionalities together make **DevRoom** an efficient and interactive platform for collaborative programming, learning, and remote teamwork.

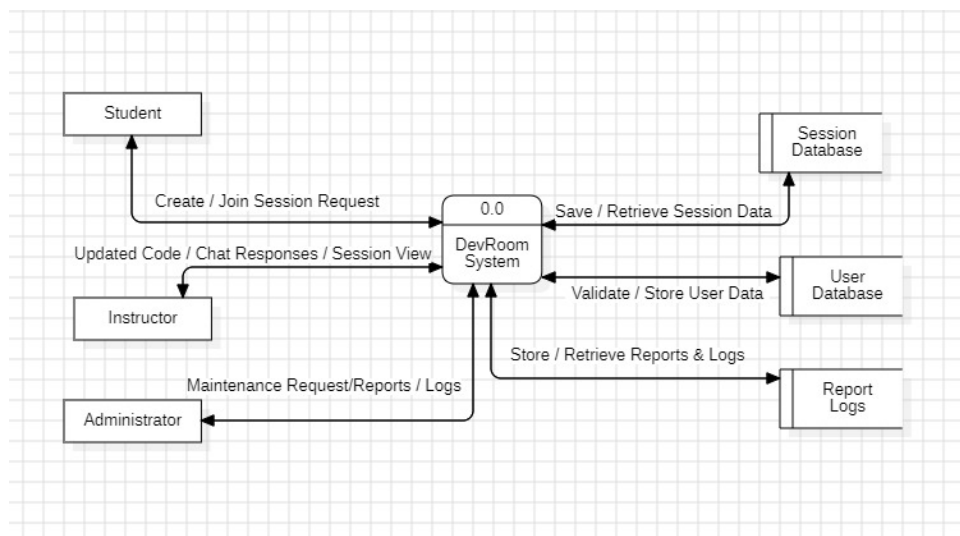


Figure 2.2: Data Flow Diagram Level 0

2.3 Users and Characteristics

2.3.1 System Administrator

The **System Administrator** is responsible for managing and maintaining the **DevRoom** platform. They hold the highest level of access privileges, allowing them to oversee all user activities, manage session data, and ensure system stability. The administrator can monitor real-time performance, handle user authentication, perform backups, and update the system when required. They are also responsible for resolving technical issues, managing server configurations, and reporting any bugs or errors to the development team.

2.3.2 Instructor

The **Instructor** is a user who utilizes the platform for teaching or demonstration purposes. They can create coding sessions, share access links with students, and guide collaborative exercises in real time. Instructors may also supervise multiple sessions, provide feedback, and use the chat and code editor features to explain programming concepts interactively.

2.3.3 Student

The **Student** is the primary user of the system and will use **DevRoom** for collaborative learning, practice, and coding exercises. Students can join sessions created by instructors or peers, participate in shared coding environments, communicate via the chat interface, and save or download their work. They are expected to use the platform frequently for educational, project-based, and self-learning purposes.

2.4 Operating environment

DevRoom is a web-based application that is platform-independent and accessible from any geographical location. The system will operate on all modern web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari, compatible with major operating systems including Windows, macOS, and Linux. It requires an active internet connection to support real-time collaboration and data synchronization. The backend server will run on a Node.js environment, while the frontend will be supported by JavaScript and related web technologies. The application is designed for **24/7 availability**, ensuring users can access it at any time, though minor service interruptions during maintenance are acceptable.

2.5 Design and Implementation Constraints

2.5.1 Hardware Constraint

DevRoom requires a laptop or desktop computer to function effectively. If these devices are unavailable, users will not be able to access or utilize the system. The platform is not optimized for mobile-based browsers, as the code editor and real-time synchronization features require larger screens and full keyboard functionality.

2.5.2 Browser Constraint

A modern web browser is required to run **DevRoom**. Users must have a compatible browser such as **Google Chrome**, **Mozilla Firefox**, **Microsoft Edge**, or **Safari** installed on their system. The browser should support the latest JavaScript standards and WebSocket technology to ensure smooth performance.

2.5.3 JavaScript Constraint

The system depends heavily on JavaScript for both frontend interactivity and real-time communication. Therefore, the browser must have JavaScript enabled and updated to its latest version; otherwise, certain features may not function correctly.

2.5.4 Internet Constraint

As **DevRoom** is a real-time collaborative web application, continuous internet connectivity is required for proper operation. Without an active internet connection, users will not be able to access sessions, synchronize code, or communicate through the chat feature.

2.5.5 Input Device Constraint

A **keyboard and mouse** are necessary input devices for interacting with the system, editing code, and navigating the interface. The application is not optimized for touchscreen-only input modes.

2.5.6 Performance Constraint

Since **DevRoom** relies on real-time synchronization, its performance is dependent on network stability and server response time. High latency or weak internet connections may cause minor delays in code updates or chat messages between users.

2.5.7 Storage Constraint

All coding sessions and chat histories are stored in the database. Therefore, storage capacity on the **MongoDB** server must be monitored and managed regularly to prevent overflow and ensure consistent performance.

2.5.8 Scalability Constraint

The system's real-time communication relies on **Socket.IO**, which may face scalability limits if a large number of users join the same session simultaneously. Server load balancing or additional optimization may be required for high-traffic usage.

2.5.9 Compatibility Constraint

DevRoom should function uniformly across major operating systems (Windows, macOS, Linux). However, compatibility issues may occur with older browsers or outdated hardware configurations that do not fully support modern JavaScript or WebSocket functionalities.

2.5.10 User Documentation

To run **DevRoom**, no advanced technical expertise is required, as the system is designed to be intuitive and user-friendly. The interface is simple and easy to navigate, allowing users such as students, instructors, and developers to participate in collaborative coding sessions without difficulty. A basic understanding of programming concepts and syntax is sufficient to use the platform effectively. The system provides on-screen guidance and tooltips to help users create or join sessions, edit code, and communicate through the built-in chat interface. Additional documentation, including a detailed user manual and setup guide, will be made available online to assist new users with account creation, session management, and troubleshooting. Below is a link to a user manual online.

Link: <https://socket.io/docs/v4/>

2.6 Assumptions and Dependencies

2.6.1 Assumptions

- It is assumed that the users of **DevRoom**—including students, instructors, and developers—possess a basic understanding of programming concepts and syntax.
- A modern web browser (such as Google Chrome, Mozilla Firefox, or Microsoft Edge) must be installed on the user’s device.
- The user’s laptop or desktop computer should be sufficiently charged or connected to a stable power source while using the application.
- Users are assumed to have access to a stable internet connection to support real-time collaboration and communication.
- It is assumed that all users will adhere to ethical collaboration practices and avoid unauthorized access or misuse of shared sessions.

2.6.2 Dependencies

The **DevRoom** web application is highly dependent on the availability of a computer system and a stable internet connection, as the platform operates entirely online. The system also relies on the proper functioning of its web server and database to ensure uninterrupted real-time collaboration. Any failure in these components or network disruption may affect the overall performance and accessibility of the software.

Chapter 3

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The DevRoom web application will feature a clean, consistent, and responsive user interface across all screens. The design will focus on usability, simplicity, and accessibility to ensure smooth interaction for all types of users. The interface will primarily consist of the following screens:

1. **Home Screen:** This is the main landing and navigation page. From here, users can create a new collaborative coding session or join an existing one using a session code or link. The screen will also include options for user login/signup, system overview, and quick instructions on how to start coding collaboratively.
2. **Editor Screen:** This is the core workspace of DevRoom, where users can write and edit code in real time. The page will include:
 - A code editor area with syntax highlighting and line numbering (powered by Monaco Editor).
 - A chat panel for communication between participants.
 - A list of connected users for identifying active collaborators.
 - Control buttons for saving sessions, downloading code, and copying the session link for sharing.
3. **Session Management Screen:** This page allows users (especially instructors or hosts) to view previously saved coding sessions, restore session data, or manage existing rooms. Users can delete, rename, or reopen saved sessions from the database.

All screens will follow a uniform color scheme, navigation layout, and typography for visual consistency. The interface will be fully responsive, adapting seamlessly to various screen resolutions and devices such as laptops and desktops.

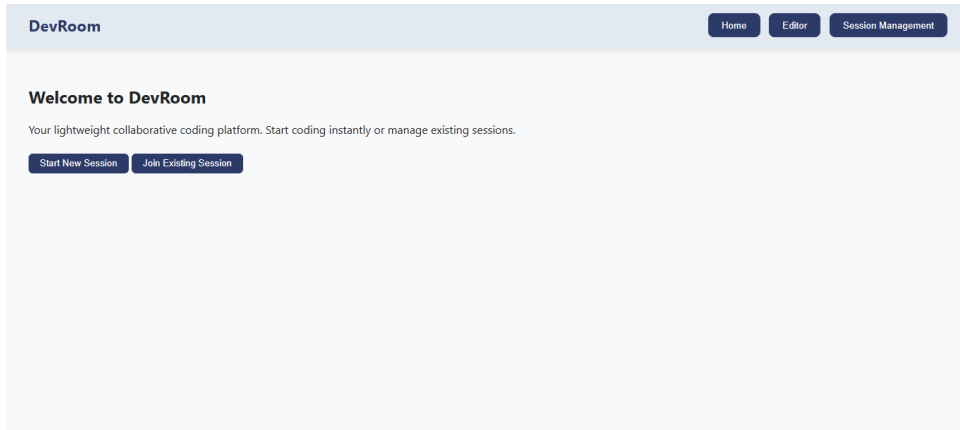


Figure 3.1: Home page

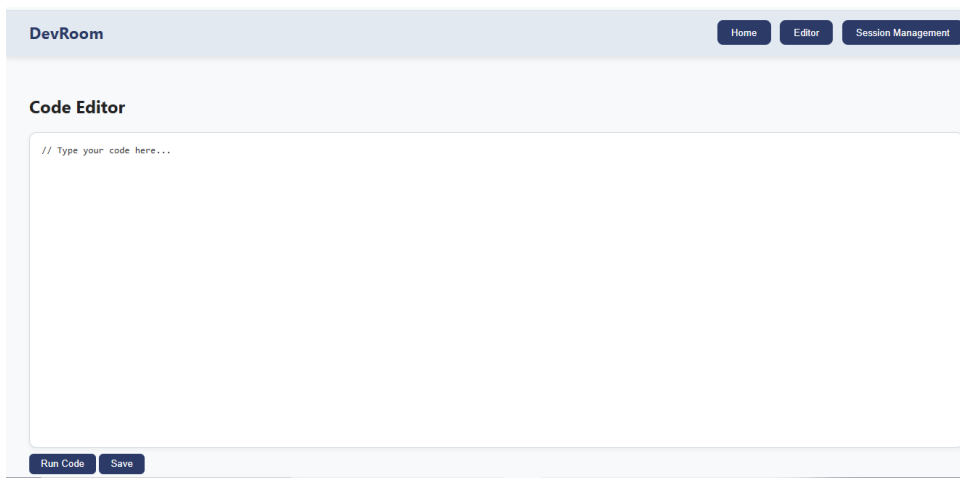


Figure 3.2: Editor page



Figure 3.3: Session page

3.1.2 Hardware Interfaces

DevRoom does not require any dedicated hardware interfaces, as it is a fully web-based application that runs on standard computing devices. The system operates through a web

browser on any desktop or laptop computer equipped with a standard keyboard, mouse, and internet connection. No additional hardware components or peripheral devices are needed for its functionality. The application relies on the user's existing hardware resources for input (keyboard, mouse) and display (monitor). Since DevRoom does not interact directly with external hardware, no special device drivers or communication libraries are required.

3.1.3 Software Interfaces

- The system will use MongoDB as the primary database service for storing session details, chat logs, and user data.
- The application backend will be developed using Node.js and Express.js frameworks to handle server-side operations and communication.
- Socket.IO will be used to establish real-time, bidirectional communication between the server and clients for live code synchronization and chat functionality.
- The system will run on commonly used operating systems such as Windows, macOS, and Linux, and will be accessed through any modern web browser.
- The frontend will utilize Monaco Editor for code editing and HTML, CSS, and JavaScript for interface design and interactivity.

3.1.4 Communications Interfaces

DevRoom operates entirely as a web-based system, requiring a stable internet connection and a modern web browser for client-server communication. The application utilizes Socket.IO to enable real-time, bidirectional data transfer between the client and the server, ensuring instant synchronization of code edits, chat messages, and session updates among users.

All communication is conducted over secure HTTPS protocols to maintain data confidentiality and integrity during transmission. Additionally, WebSocket technology is used under Socket.IO to provide low-latency and continuous connections for real-time collaboration. The system ensures that sensitive data such as login credentials and session information are encrypted before transmission, following standard web security practices such as TLS (Transport Layer Security).

3.2 Functional requirements

3.2.1 Main Page

User Requirement

1. The user will access the front (home) page of DevRoom through a web browser.

System Requirement

1. The home page will display options to either:

- (a) Create a new coding session.
 - (b) Join an existing session using a session ID or invitation link.
2. Upon selecting either option, the user will be redirected to the collaborative code editor page.
3. The system will verify active internet connectivity before allowing access to the session workspace.

3.2.2 Code Editor Page

User Requirement

1. The user can write, edit, and view code collaboratively with others in real time.
2. The user can communicate with other participants through a built-in chat box.

System Requirement

1. The system will provide a code editor interface with syntax highlighting and live code synchronization using Socket.IO.
2. A chat panel will be displayed alongside the editor for communication.
3. The system will display the list of active participants in the session.
4. All changes made by any user will be reflected instantly for all connected participants.

3.2.3 Save and Load Session

User Requirement

1. The user can save the current session for future use.
2. The user can reopen a previously saved session.

System Requirement

1. The system will allow session data (including code and chat history) to be stored in the MongoDB database.
2. Users can retrieve and reload saved sessions upon request.
3. A "Save Session" and "Load Session" button will be available in the editor toolbar.

3.2.4 Download Code

User Requirement

1. The user can download their current code for offline use.

System Requirement

1. The system will provide a Download Code button that allows users to export their current code as a .txt, .js, or .py file.
2. The downloaded file will include the latest code version from the active session.

3.2.5 Chat Functionality

User Requirement

1. The user can send and receive text messages in real time.

System Requirement

1. The chat interface will allow live message exchange between all participants.
2. The system will display usernames with timestamps for clarity.
3. Messages will be broadcast across all connected by Socket.IO.

Table 3.1: Function 1: Create or Join Session

Requirement # 1	
Function description	Allows a user to start a new collaborative coding session or join an existing one using a session ID or invite link.
Input and origin	Takes user input from the home page form to specify a session name for a new session or a session ID/link to join an existing one.
Output and destination	Redirects user to the collaborative editor workspace.
Action to be taken	The system validates the session information and connects the user to the real-time collaboration environment.
Pre-condition	User must have an active internet connection and a supported web browser.
Post-condition	A new session is created or the user is successfully connected to an existing one.
Side effects	Invalid session ID or server connectivity issue prevents session access.

Table 3.2: Function 2: Real-Time Code Editing

Requirement # 2	
Function description	Enables multiple users to collaboratively write and edit code in real time within a shared editor.
Input and origin	Captures code text entered by any participant through user keyboard input in the Monaco Editor.
Output and destination	Updated code displayed instantly across all connected users' editors.
Action to be taken	The system captures keystroke and broadcasts updates through Socket.IO for live synchronization.
Pre-condition	Active coding session established and users connected to the same room.
Post-condition	All users see the same updated code in real time.
Side effects	Internet disconnection or server delay may cause temporary desynchronization.

Table 3.3: Function 3: Chat Communication

Requirement # 3	
Function description	Allows users to exchange text messages within an active coding session.
Input and origin	Captures chat message text entered by users through the chat interface.
Output and destination	Message appears in all participants' chat panels with username and timestamp.
Action to be taken	The message is transmitted via Socket.IO and displayed to all connected clients.
Pre-condition	Users are connected to an active session.
Post-condition	Message is displayed for all active participants.
Side effects	Network loss or server downtime causes delayed or missing messages.

Table 3.4: Function 4: Save and Load Session

Requirement # 4	
Function description	Enables users to save their coding session and reload it later to continue their work.
Input and origin	Captures user interaction with the editor toolbar when selecting the "Save" or "Load" button.
Output and destination	Confirmation of saved session or restored previous session.
Action to be taken	The system stores or retrieves session data (code, chat history) from MongoDB.
Pre-condition	User is connected to an active session with database access.
Post-condition	Session data is stored or successfully loaded.
Side effects	Database error or connectivity failure prevents save/load operation.

Table 3.5: Function 5: Download Code

Requirement # 5	
Function description	Allows users to download their current code file for offline access.
Input and origin	Captures user action in the code editor toolbar when clicking the "Download Code" button.
Output and destination	Code file downloaded in .txt .js, or py format.
Action to be taken	The system retrieves the latest code and triggers a file download.
Pre-condition	Active session with unsaved code available.
Post-condition	User obtains a local copy of their code.
Side effects	Browser download permission denied or file creation error.

3.3 Behaviour Requirements

3.3.1 Use case View

Code Editor Use Case

Once redirected to the collaborative editor page, users can write, edit, and view code in real time. Multiple participants can simultaneously modify the same file, and all changes are instantly synchronized using Socket.IO. This use case further extends into chat communication, saving/loading sessions, and downloading code files.

Chat Communication Use Case

Within the editor, users can send and receive text messages through the integrated chat panel. This facilitates discussion and teamwork during live sessions. The chat messages are displayed with timestamps and usernames, ensuring clarity and coordination among participants.

Session Management Use Case

This use case allows users (especially instructors and administrators) to manage coding sessions by saving, loading, or deleting previous work. The system retrieves stored session data from MongoDB and restores it for continued use.

Administration Use Case

The Administrator monitors system performance, manages database integrity, and ensures stable server operation. This use case extends to tasks such as system updates, managing user load, and monitoring active sessions to maintain performance.

System Report Use Case

Periodic system usage reports are generated for the Administrator and Developers. These reports summarize user activity, active sessions, and potential performance issues, helping to maintain transparency and improve the platform's reliability.

Error Handling and Maintenance Use Case

In case of technical faults, connection errors, or bugs, this use case manages issue detection and resolution. It involves actions like logging errors, notifying the admin, and updating or debugging the application to ensure continuous availability and stability.

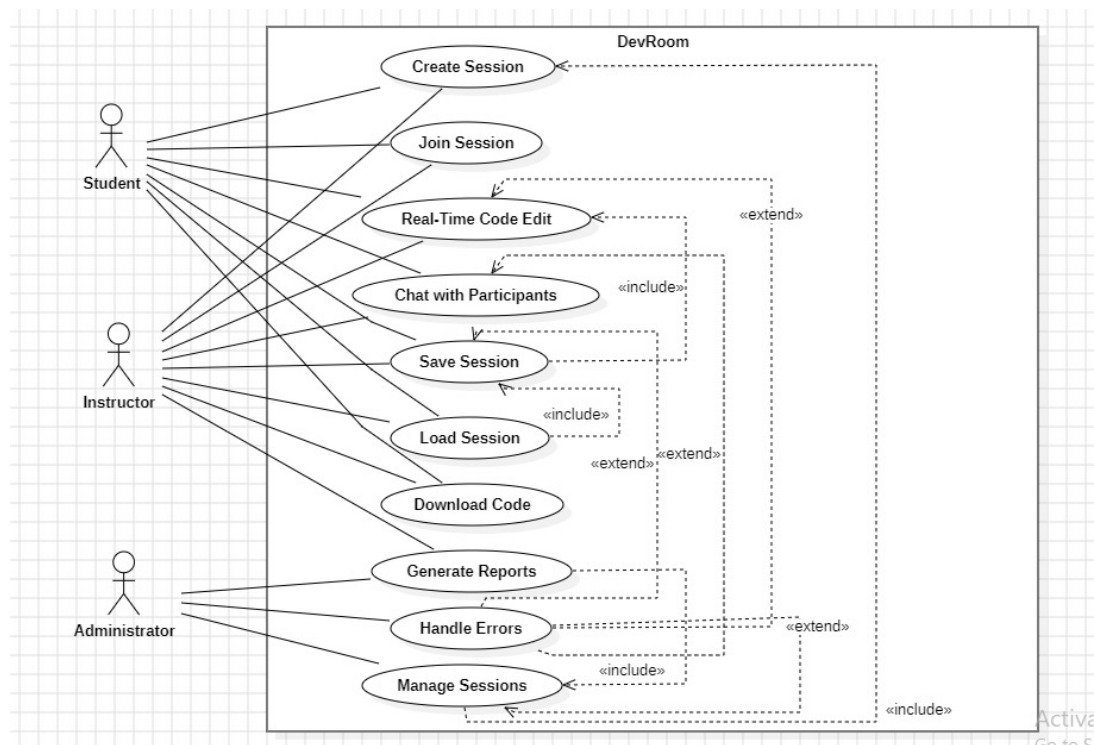


Figure 3.4: Use Case Diagram

Chapter 4

Other Non-Functional Requirements

4.1 Performance Requirements

Since DevRoom is an interactive, real-time collaborative coding platform, its primary performance requirement is low latency and fast response time to ensure smooth collaboration between multiple users.

1. When a user creates or joins a coding session, the workspace should load within 3 to 4 seconds.
2. Any code updates made by one participant should be reflected on all other connected users' screens instantaneously (with a maximum delay of 1 second).
3. Chat messages should be transmitted and displayed to all participants with minimal delay (under 1 second) to maintain effective communication.
4. Saving or loading a session from the database should not take more than 5 seconds, depending on the network and data size.
5. During high traffic or multiple concurrent sessions, the system should maintain stable performance without noticeable lag or dropped connections.
6. Opening or closing new sessions, accessing stored data, or reconnecting after temporary disconnection should occur within 2 seconds to ensure seamless user experience.

4.2 Safety and Security Requirements

4.2.1 Safety

Although DevRoom is a web-based software and does not involve any physical hazards, certain safety measures are established to ensure the protection of user data, system reliability, and a safe online experience:

1. **Data Backup Safety:** The system must perform regular backups of session data and chat history to prevent data loss in case of unexpected system failure or server downtime.

2. **Session Integrity Safety:** The system should automatically save coding progress every 30 seconds to minimize data loss caused by sudden disconnections or browser crashes.
3. **Server Recovery Safety:** In the event of a critical failure or server error, the system should be capable of restoring data from the latest backup to ensure service continuity without significant loss.

4.2.2 Security

DevRoom handles real-time collaborative communication, making it essential to ensure secure access, data protection, and system integrity. The following security mechanisms will be applied:

1. **Secure Communication:** All data transmission between client and server will occur over encrypted HTTPS and secure WebSocket protocols to prevent eavesdropping or data interception.
2. **User Authentication:** Users must log in using a verified account (email or institutional credentials) before joining or creating sessions, ensuring that only authorized users participate.
3. **Access Control:** Only system administrators will have privileges to manage, update, or delete system configurations. Regular users will be restricted to session-based operations.
4. **Data Protection:** Sensitive data such as user credentials and session tokens will be stored using encryption and will not be exposed to other users.
5. **Server Security:** The backend server must be protected with firewall configurations, strong password policies, and periodic software updates to prevent unauthorized modifications or attacks.
6. **Session Security:** Each collaborative session will be assigned a unique and randomly generated access code to prevent unauthorized entry.

4.3 Software Quality Attributes

4.3.1 Adaptability

DevRoom is designed to accommodate future changes in stored data or session information within the database. The system architecture allows easy modification and scaling of features without major structural changes.

4.3.2 Availability

The system will be available online for users most of the time, ensuring continuous access. Scheduled maintenance or minor downtime should not exceed a few minutes to minimize inconvenience to users.

4.3.3 Correctness

All collaborative and synchronization operations must execute accurately to ensure that every user sees the same, consistent version of the shared code in real time.

4.3.4 Flexibility

The software can be modified in the future to add or remove functionalities such as new programming language modes, additional collaboration tools, or enhanced security features as per user requirements.

4.3.5 Interoperability

Data is efficiently transferred between the MongoDB database, web server, and client browsers. This ensures smooth communication between system components for reliable real-time collaboration.

4.3.6 Maintainability

The software is developed using modular coding practices, allowing easy debugging, maintenance, and enhancement whenever system faults or update.

4.3.7 Portability

DevRoom can be accessed from any device with a web browser and internet connection, including systems running Windows, macOS, or Linux. It does not require complex installation or configuration.

4.3.8 Reliability

The system must remain stable and consistent regardless of the number of users connected simultaneously. Each action performed by a user should produce correct and predictable results.

4.3.9 Reuseability

The current version of DevRoom can serve as the foundation for future upgrades. Modules such as the code editor, chat system, and session handler can be reused with minimal modification in extended versions.

4.3.10 Robustness

The software should handle invalid inputs gracefully. Proper validation and error-handling mechanisms will prevent crashes and maintain system stability under unexpected conditions.

4.3.11 Testability

Each module of the system—including real-time code synchronization, chat, and database operations—can be tested independently to verify correct functionality, low response time, and reliability.

4.3.12 Useability

The user interface of DevRoom is designed to be clean, intuitive, and easy to use. Both beginners and experienced programmers should be able to navigate and utilize the system without difficulty or prior training.

Chapter 5

Other Requirements

There are no additional requirements beyond those specified in previous sections. All functional, safety, and quality attributes have been sufficiently addressed within the scope of this document.

Chapter 6

Appendix A - Data Dictionary

The Data Dictionary defines all constants, variables, inputs, outputs, and functional states referenced in the DevRoom system. It serves as a central reference to ensure clarity in data management, variable naming, and software behavior. Each entry includes its description, data type, related operations, and applicable constraints.

6.1 System Constants

Table 6.1: System Constants

Constant Name	Description	Type / Range	Related Operations / Requirements
max_users	Maximum number of concurrent users per session	Integer (Default = 10)	Session creation and connection validation
max_file_size	Maximum allowed code file size in bytes	Integer (1 MB)	Code save and update functions
default_lang	Default programming language for new sessions	String ("JavaScript")	Used in session initialization
max_message_len	Maximum characters allowed in chat message	Integer (255)	Chat input validation
auto_save_interval	Time interval for automatic code saving	Integer (30 sec)	Auto-save background task
report_retent_D	Days to store system reports	Integer (30)	Log cleanup and maintenance

6.2 State Variables

Table 6.2: State Variables

Variable Name	Possible States	Description	Module / Function
user_status	{active, inactive, disconnected}	Current connection state of user	Socket Manager
session_state	{open, ongoing, closed}	Indicates whether a session is active or ended	Session Controller
editor_state	{editing, saving, idle, error}	Real-time state of the code editor	Code Editor Component
message_state	{sending, delivered, failed}	Delivery status of chat messages	Messaging Module
system_mode	{maintenance, live, error}	System's operational state	Admin Dashboard
connection_status	{connected, reconnecting, disconnected}	Network connection state	Socket.IO Handler

6.3 Input Variables

Table 6.3: Input Variables

Input Name	Source	Data Type / Format	Description	Related Requirement
username	User	String	Name entered during registration	User login requirement
email	User	String	Email entered during sign-up	Validation requirement
password	User	String	Password for authentication	Security requirement
session_name	Instructor / Student	String	Title for new session	Session creation function
language_mode	User	String	Programming language selected	Editor setup requirement
message_text	User	String	Chat message entered by user	Chat input handling
code_content	User	Text	Code typed in the editor	Core system functionality

6.4 Output Variables

Table 6.4: Output Variables

Output Name	Destination	Type / Format	Description	Generated From / By
login_response	User Interface	JSON	Confirms login success or failure	Authentication API
session_summary	Instructor / Admin	JSON / Text	Summary of users, edits, and duration	Session Module
code_preview	UI Editor	Text	Displays formatted code in the editor	Rendering Module
chat_output	Chat Window	Text	Displays chat messages in real-time	Messaging Subsystem
error_alert	User Interface	String	Notifies user of invalid operations	Error Handler
system_report	Admin Dashboard	JSON / PDF	Logs and reports of system activity	Maintenance Subsystem

6.5 Functional Data (Key Operations)

Table 6.5: Functional Data and Key Operations

Operation	Input(s)	Output(s)	Description / Purpose
createSession()	session_name, language_mode	session_id	Creates a new collaborative coding room
joinSession()	session_id, user_id	connection_status	Connects user to an existing session
sendMessage()	sender_id, message_text	message_state	Sends chat message to participants
updateCode()	code_content	editor_state	Updates shared code in real time
generateReport()	session_id, report_type	report_file	Generates performance or activity report
saveSession()	session_id, code_content	confirmation_status	Saves the current code to database
closeSession()	session_id	session_state	Ends session and updates database

6.6 Notes

- All data variables follow *camelCase* naming convention.
- System constants are defined server-side and cannot be modified by clients.
- Input and output variables are validated both client-side (JavaScript) and server-side (Node.js).
- Future versions of DevRoom may include additional state variables for voice and video collaboration features.